

第7章 贪心法

7.1 贪心法概述

7.2 求解活动安排问题

7.3 求解背包问题

7.4 求解最优装载问题

7.5 求解田忌赛马问题

7.6 求解多机调度问题

7.7 哈夫曼编码

7.8 求解流水作业调度问题

7.1 贪心法概述

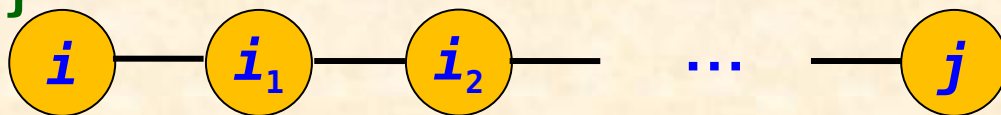
7.1.1 什么是贪心法

贪心法的**基本思路**是在对问题求解时总是做出在当前看来是最好的选择，也就是说贪心法不从整体最优上加以考虑，所做出的仅是在某种意义上的局部最优解。

人们通常希望找到整体最优解，所以采用贪心法**需要证明**设计的算法确实是整体最优解或求解了它要解决的问题。

例如，求解一个带权无向图 G 中从顶点 i 到顶点 j ($i \neq j$) 的最短路径，可以分析出这样的最短路径一定是简单路径，所以约束条件为：

求解的路径为 $\{ (i, i_1), (i_1, i_2), \dots, (i_m, j) \mid (i, i_1), (i_1, i_2), \dots, (i_m, j) \text{ 均为图 } G \text{ 的边, 且 } i_k (1 \leq k \leq m) \text{ 均不相同} \}$



目标函数就是要使这样的路径最短，即：

$$\begin{aligned} \text{MIN}_{\text{pathlength}} \{ & (i, i_1), (i_1, i_2), \dots, (i_m, j) \mid \\ & \text{pathlength} = w(i, i_1) + w(i_1, i_2) + \dots \\ & + w(i_m, j), \\ & w(i, k) \text{ 表示 } (i, k) \text{ 的权值} \} \end{aligned}$$

贪心法从问题的某一个初始解 $\{ \}$ 出发, 采用逐步构造最优解的方法向给定的目标前进, 每一步决策产生 n -元组解 $(x_0, x_1, \dots, x_{n-1})$ 的一个分量。

贪心法每一步上用作决策依据的选择准则被称为最优量度标准 (或贪心准则), 也就是说, 在选择解分量的过程中, 添加新的解分量 x_k 后, 形成的部分解 $(x_0, x_1, \dots, x_k)$ 不违反可行解约束条件。

每一次贪心选择都将所求问题简化为规模更小的子问题, 并期望通过每次所做的局部最优选择产生出一个全局最优解。

7.1.2 贪心法求解的问题应具有的性质

1. 贪心选择性 质

所谓贪心选择性是指所求问题的整体最优解可以通过一系列局部最优的选择，即贪心选择来达到。

也就是说，贪心法仅在当前状态下做出最好选择，即局部最优选择，然后再去求解做出这个选择后产生的相应子问题的解。

2. 最优子结构性质

如果一个问题的最优解包含其子问题的最优解，则称此问题具有**最优子结构性质**。

问题的最优子结构性质是该问题可用动态规划算法或贪心法求解的**关键特征**。

7.1.3 贪心法的一般求解过程

贪心法求解问题的算法框架如下:

```
SolutionType Greedy(SType a[],int n)
```

```
// 假设解向量  $(x_0, x_1, \dots, x_{n-1})$  类型为 SolutionType，其分量为 SType 类型
```

```
{ SolutionType x={};                                // 初始时，解向量不包含任何分量
```

```
    for (int i=0;i<n;i++)                                // 执行 n 步操作
```

```
    { SType xi=Select(a);                                // 从输入 a 中选择一个当前最好的分量
```

```
        if (Feasible(xi))                                // 判断  $x_i$  是否包含在当前解中
```

```
            solution=Union(x,xi);                        // 将  $x_i$  分量合并形成 x
```

```
    }
```

```
    return x;                                // 返回生成的最优解
```

```
}
```


7.2 求解活动安排问题

【问题描述】 假设有一个需要使用某一资源的 n 个活动所组成的集合 S , $S=\{1, \dots, n\}$ 。该资源任何时刻只能被一个活动所占用, 活动 i 有一个开始时间 b_i 和结束时间 e_i ($b_i < e_i$), 其执行时间为 $e_i - b_i$, 假设最早活动执行时间为 0。

一旦某个活动开始执行, 中间不能被打断, 直到其执行完毕。若活动 i 和活动 j 有 $b_i \geq e_j$ 或 $b_j \geq e_i$, 则称这两个活动**兼容**。

设计算法求一种最优活动安排方案, 使得**所有安排的活动个数最多**。

【问题求解】 假设活动时间的参考原点为 0。一个活动 i ($1 \leq i \leq n$) 用一个区间 $[b_i, e_i)$ 表示, 当活动按结束时间 (右端点) 递增排序后, 两个活动 $[b_i, e_i)$ 和 $[b_j, e_j)$ 兼容 (满足 $b_i \geq e_j$ 或 $b_j \geq e_i$) 实际上就是指它们不相交。

用数组 A 存放所有的活动, $A[i].b$ ($1 \leq i \leq n$), 存放活动起始时间, $A[i].e$ 存放活动结束时间。

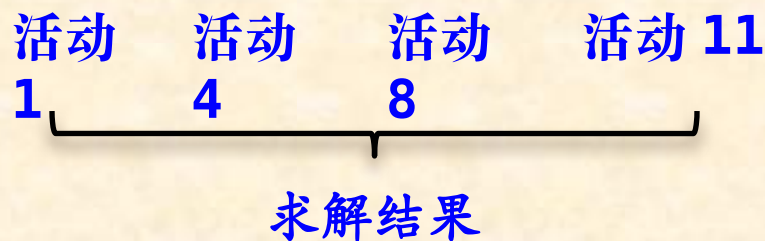
例如，对于下表的 $n=11$ 个活动（已按结束时间递增排序） A ：

i	1	2	3	4	5	6	7	8	9	10	11
开始时间	1	3	0	5	3	5	6	8	8	2	12
结束时间	4	5	6	7	8	9	10	11	12	13	15

产生最大兼容活动集合的过程：

活动 1	✓
活动 2	×
活动 3	×
活动 4	✓
活动 5	×
活动 6	×
活动 7	×
活动 8	✓
活动 9	×
活动 10	×
活动 11	✓

最大兼容活动集合：



// 问题表示

```
struct Action          // 活动的类型声明
{   int b;             // 活动起始时间
    int e;             // 活动结束时间
    bool operator<(const Action &s) const // 重载 < 关系函数
{
    return e<=s.e;     // 用于按活动结束时间递增排序
}
};
int n=11;
Action A[]={ {0}, {1,4}, {3,5}, {0,6}, {5,7}, {3,8}, {5,9}, {6,10},
{8,11},
            {8,12}, {2,13}, {12,15}}; // 下标 0 不用
```

// 求解结果表示

```
bool flag[MAX];        // 标记选择的活动
int Count=0;           // 选取的兼容活动个数
```

```
void solve()
```

```
{  memset(flag,0,sizeof(flag));
```

```
    sort(A+1,A+n+1);
```

```
    int preend=0;
```

```
    for (int i=1;i<=n;i++)
```

```
    {  if (A[i].b>=preend)
```

```
        {  flag[i]=true;
```

```
            preend=A[i].e;
```

```
        }
```

```
    }
```

```
}
```

```
// 求解最大兼容活动子集
```

```
// 初始化为 false
```

```
//A[1..n] 按活动结束时间递增排序
```

```
// 前一个兼容活动的结束时间
```

```
// 扫描所有活动
```

```
// 找到一个兼容活动
```

```
// 选择 A[i] 活动
```

```
// 更新 preend 值
```

【算法分析】 算法的主要时间花费在排序上，排序时间为 $O(n\log_2 n)$ ，所以整个算法的时间复杂度为 $O(n\log_2 n)$ 。

【算法证明】通常证明一个贪心选择得出的解是最优解的一般的方法是，构造一个初始最优解，然后对该解进行修正，使其第一步为一个贪心选择，证明总是存在一个以贪心选择开始的求解方案。

对于本问题，所有活动按结束时间递增排序，就是要证明：若 X 是活动安排问题 A 的最优解， $X = X' \cup \{1\}$ ，则 X' 是 $A' = \{i \in A : e_i \geq b_1\}$ 的活动安排问题的最优解。

首先证明总存在一个以活动 1 开始的最优解。

如果第一个选中的活动为 k ($k \neq 1$)，可以构造另一个最优解 Y ， Y 中的活动是兼容的， Y 与 X 的活动数相同。

那么用活动 1 取代活动 k 得到 Y' ，因为 $e_1 \leq e_k$ ，所以 Y' 中的活动是兼容的，即 Y' 也是最优的，这就说明总存在一个以活动 1 开始的最优解。

当做出了对活动 1 的贪心选择后，原问题就变成了在活动 2、...、 n 中找与活动 1 兼容的那些活动的子问题。亦即，如果 X 为原问题的一个最优解，则 $X' = X - \{1\}$ 也是活动选择问题 $A' = \{i \in A \mid b_i \geq e_1\}$ 的一个

反证法：如果能找到一个 A' 的含有比 X' 更多活动的解 Y' ，则将活动 1 加入 Y' 后就得到 A 的一个包含比 X 更多活动的解 Y ，这就与 X 是最优解的假设相矛盾。

因此，在每一次贪心选择后，留下的是一个与原问题具有相同形式的最优化问题，即最优子结构性质。

【例 7.2】 求解蓄栏保留问题。农场有 n 头牛，每头牛会有一个特定的时间区间 $[b, e]$ 在蓄栏里挤牛奶，并且一个蓄栏里任何时刻只能有一头牛挤奶。

现在农场主希望知道最少蓄栏能够满足上述要求，并给出每头牛被安排的方案。对于多种可行方案，输出一种即可。

解：牛的编号为 $1 \sim n$ ，每头牛的挤奶时间相当于一个活动，与前面活动安排问题不同，这里的活动时间是闭区间，例如 $[2, 4]$ 与 $[4, 7]$ 是交叉的，它们不是兼容活动。

采用与求解活动安排问题类似的贪心思路，将所有活动这样排序：结束时间相同按开始时间递增排序，否则按结束时间递增排序。

求出一个最大兼容活动子集，将它们安排在一个蓄栏中（蓄栏编号为 1）；如果没有安排完，再在剩余的活动再求下一个最大兼容活动子集，将它们安排在另一个蓄栏中（蓄栏编号为 2），以此类推。

也就是说，最大兼容活动子集的个数就是最少蓄栏个数。

<i>i</i>	1	2	3	4	5	6	7
开始时间	1	2	5	8	4	12	11
结束时间	4	5	7	9	10	13	15



1	3	4	6	2	7	5
1	5	8	12	2	11	4
4	7	9	13	5	15	10



最大兼容活动子集个数为 3

// 问题表示

```
struct Cow
```

```
{  int no;
```

```
    int b;
```

```
    int e;
```

```
    bool operator<(const Cow &s) const    // 重载 < 关系函数
```

```
    {  if (e==s.e)                        // 结束时间相同按开始时间递增排序
```

```
        return b<=s.b;
```

```
        else
```

```
        return e<=s.e;
```

```
    }
```

```
};
```

```
int n=5;
```

```
Cow A[]={ {0}, {1,1,10}, {2,2,4}, {3,3,6}, {4,5,8}, {5,4,7} };
```

```
// 下标 0 不用
```

// 求解结果表示

```
int ans[MAX];
```

```
//ans[i] 表示第 A[i].no 头牛的蓄栏编号
```

```

void solve()
{
    sort(A+1,A+n+1);
    memset(ans,0,sizeof(ans));
    int num=1;

    for (int i=1;i<=n;i++)
    {
        if (ans[i]==0)
        {
            ans[i]=num;
            int preend=A[i].e;

            for (int j=i+1;j<=n;j++) // 查找一个最大兼容活动子集
            {
                if (A[j].b>preend && ans[j]==0)
                {
                    ans[j]=num; // 将兼容活动子集中活动安排在 num 蓄栏中
                    preend=A[j].e; // 更新结束时间
                }
            }
            num++; // 查找下一个最大兼容活动子集 , num 增 1
        }
    }
}

```

// 求解最大兼容活动子集个数

//A[1..n] 按指定方式排序

// 初始化为 0

// 蓄栏编号

//i、j 均为排序后的下标

// 第 i 头牛还没有安排蓄

// 第 i 头牛安排蓄栏 num

// 前一个兼容活动的结束

栏

时间


```
void main()
{   solve();
    printf(" 求解结果 \n");
    for (int i=1;i<=n;i++)
        printf("      牛 %d 安排的蓄栏 : %d\n",A[i].no,ans[i]);
}
```

<i>i</i>	1	2	3	4	5
开始时间	1	2	3	5	4
结束时间	10	4	6	8	7



求解结果

牛 2 安排的蓄栏 :1

牛 3 安排的蓄栏 :2

牛 5 安排的蓄栏 :3

牛 4 安排的蓄栏 :1

牛 1 安排的蓄栏 :4

7.3 求解背包问题

【问题描述】 设有编号为 1、2、...、 n 的 n 个物品，它们的重量分别为 w_1 、 w_2 、...、 w_n ，价值分别为 v_1 、 v_2 、...、 v_n ，其中 w_i 、 v_i ($1 \leq i \leq n$) 均为正数。

有一个背包可以携带的最大重量不超过 W 。求解**目标**：在不超过背包负重的前提下，使背包装入的总价值最大（即效益最大化），与 0/1 背包问题的区别是，这里的每个物品可以取一部分装入背包。

问题求解： 这里采用贪心法求解。设 x_i 表示物品 i 装入背包的情况， $0 \leq x_i \leq 1$ 。根据问题的要求，有如下约束条件和目标函数：

$$\sum_{i=1}^n w_i x_i \leq W \quad 0 \leq x_i \leq 1 \quad (1 \leq i \leq n)$$

$$\text{MAX} \left\{ \sum_{i=1}^n v_i x_i \right\}$$

于是问题归结为寻找一个满足上述约束条件，并使目标函数达到最大的解向量 $X = \{x_1, x_2, \dots, x_n\}$ 。

例如, $n=3$, $(w_1, w_2, w_3)=(18, 15, 10)$, $(v_1, v_2, v_3)=(25, 24, 15)$, $W=20$, 其中的 4 个可行解如下:

解编号	(x_1, x_2, x_3)	$\sum_{i=1}^n w_i x_i$	$\sum_{i=1}^n v_i x_i$
①	$(1/2, 1/3, 1/4)$	16.5	24.25
②	$(1, 2/15, 0)$	20	28.2
③	$(0, 2/3, 1)$	20	31

在这 4 个可行解中, 第④个解的效益最大, 可以求出它是这个背包问题的最优解。 $(0, 1, 1/2)$ 20 31.5

贪心策略：选择单位重量价值最大的物品。

每次从物品集合中选择单位重量价值最大的物品，如果其重量小于背包容量，就可以把它装入，并将背包容量减去该物品的重量，然后就面临了一个最优子问题——它同样是背包问题，只不过背包容量减少了，物品集合减少了。

因此背包问题具有最优子结构性质。

对于下表一个背包问题, $n=5$, 设背包容量 $W=100$, 其求解过程如下.

i	1	2	3	4	5
w_i	10	20	30	40	50
v_i	20	30	66	40	60
v_i/w_i	2.0	1.5	2.2	1.0	1.2

(1) 将单位价值即 v/w 递减排序, 其结果为 $\{66/30, 20/10, 30/20, 60/50, 40/40\}$, 物品重新按 1 ~ 5 编号。

i	1	2	3	4	5
w_i	30	10	20	50	40
v_i	66	20	30	60	40
v_i/w_i	2.2	2.0	1.5	1.2	1.0

(2) 设背包余下装入的重量为 $weight=W$ 。

i	1	2	3	4	5
w_i	30	10	20	50	40
v_i	66	20	30	60	40
v_i/w_i	2.2	2.0	1.5	1.2	1.0

(3) 从 $i=1$ 开始, $w[1]<\text{weight}$ 成立, 表明物品 1 能够装入, 将其装入到背包中, 置 $x[1]=1$, $\text{weight}=\text{weight}-w[1]=70$, i 增 1 即 $i=2$ 。

$w[2]<\text{weight}$ 成立, 表明物品 2 能够装入, 将其装入到背包中, 置 $x[2]=1$, $\text{weight}=\text{weight}-w[2]=60$, i 增 1 即 $i=3$ 。

$w[3]<\text{weight}$ 成立, 表明物品 3 能够装入, 将其装入到背包中, 置 $x[3]=1$, $\text{weight}=\text{weight}-w[3]=50$, i 增 1 即 $i=4$ 。

$w[4]<\text{weight}$ 不成立, 且 $\text{weight}>0$, 表明只能将物品 4 部分装入, 装入比例 $=\text{weight}/w[4]=50/60=80\%$, 置 $x[4]=0.8$ 。

算法结束, 得到 $X=\{1, 1, 1, 0.8, 0\}$ 。

// 问题表示

```
int n=5;
```

```
double W=100;
```

// 限重

```
struct NodeType
```

```
{ double w;
```

```
  double v;
```

```
  double p;
```

// $p=v/w$

```
  bool operator<(const NodeType &s) const
```

```
{
```

```
    return p>s.p;
```

// 按 p 递减排序

```
}
```

```
};
```

```
NodeType A[]={ {0}, {10,20}, {20,30}, {30,66}, {40,40}, {50,60} };
```

// 下标 0 不用

// 求解结果表示

```
double V;
```

// 最大价值

```
double x[MAXN];
```

void Knap()	// 求解背包问题并返回总价值
{ V=0;	//V 初始化为 0
double weight=W;	// 背包中能装入的余下重量
memset(x,0,sizeof(x));	// 初始化 x 向量
int i=1;	
 while (A[i].w<weight)	// 物品 i 能够全部装入时循环
{ x[i]=1;	// 装入物品 i
weight-=A[i].w;	// 减少背包中能装入的余下重量
V+=A[i].v;	// 累计总价值
i++;	// 继续循环
}	
 if (weight>0)	// 当余下重量大于 0
{ x[i]=weight/A[i].w;	// 将物品 i 的一部分装入
V+=x[i]*A[i].v;	// 累计总价值
}	
}	

```
void main()
{   printf(" 求解过程 \n");
    for (int i=1;i<=n;i++)           // 求 v/w
        A[i].p=A[i].v/A[i].w;
    printf("(1) 排序前 \n");dispA();

    sort(A+1,A+n+1);                 //A[1..n] 排序

    printf("(2) 排序后 \n"); dispA();

    Knap();

    printf("(3) 求解结果 \n");       // 输出结果
    printf("      x:  [");
    for (int j=1;j<=n;j++)
        printf("%g, ",x[j]);
    printf("%g]\n",x[n]);
    printf("      总价值 =%g\n",V);
}
```

【算法证明】 假设对于 n 个物品，按 v_i/w_i ($1 \leq i \leq n$) 值递减排序得到 1、2、...、 n 的序列，即 $v_1/w_1 \geq v_2/w_2 \geq \dots \geq v_n/w_n$ 。设 $X = \{x_1, x_2, \dots, x_n\}$ 是本算法找到解。

如果所有的 x_i 都等于 1，这个解明显是最优解。否则，设 $minj$ 是满足 $x_{minj} < 1$ 的最小下标。考虑算法的工作方式， $\sum_{i=1}^n v_i x_i$ 很明显，当 $i < minj$ 时， $x_i = 1$ ，当 $i > minj$ 时， $x_i = 0$ ，并且。设 X 的值为 $V(X) =$

设 $Y = \{y_1, y_2, \dots, y_n\}$ 是该背包问题的一个最优可行解，因此有 $\sum_{i=1}^n w_i y_i \leq W$ ，
 以而有 $\sum_{i=1}^n w_i (x_i - y_i) = \sum_{i=1}^n w_i x_i - \sum_{i=1}^n w_i y_i \geq 0$ ，这个解的值为

$$V(Y) = \sum_{i=1}^n v_i y_i \quad \text{则}$$

$$\frac{V(X) - V(Y)}{V(Y)} = \sum_{i=1}^n v_i (x_i - y_i) = \sum_{i=1}^n w_i \frac{v_i}{w_i} (x_i - y_i)$$

当 $i < \min j$ 时, $x_i = 1$, 所以 $x_i - y_i \geq 0$, 且 $v_i/w_i \geq v_{\min j}/w_{\min j}$ 。

当 $i > \min j$ 时, $x_i = 0$, 所以 $x_i - y_i \leq 0$, 且 $v_i/w_i \leq v_{\min j}/w_{\min j}$ 。

当 $i = \min j$ 时, $v_i/w_i = v_{\min j}/w_{\min j}$ 。

$$\begin{aligned} \text{则 } V(X) - V(Y) &= \sum_{i=1}^n w_i \frac{v_i}{w_i} (x_i - y_i) = \sum_{i=1}^{\min j - 1} w_i \frac{v_i}{w_i} (x_i - y_i) + \sum_{i=\min j}^{\min j} w_i \frac{v_i}{w_i} (x_i - y_i) + \sum_{i=\min j + 1}^n w_i \frac{v_i}{w_i} (x_i - y_i) \\ &> \sum_{i=1}^{\min j - 1} w_i \frac{v_{\min j}}{w_{\min j}} (x_i - y_i) + \sum_{i=\min j}^{\min j} w_i \frac{v_{\min j}}{w_{\min j}} (x_i - y_i) + \sum_{i=\min j + 1}^n w_i \frac{v_{\min j}}{w_{\min j}} (x_i - y_i) \\ &= \frac{v_{\min j}}{w_{\min j}} \sum_{i=1}^n w_i (x_i - y_i) \geq 0 \end{aligned}$$

这样与 Y 是最优解的假设矛盾, 也就是说没有哪个可行解的价值会大于 $V(X)$, 因此解 X 是最优解。

【算法分析】 排序的时间复杂性为 $O(n\log_2 n)$ ，while 循环的时间为 $O(n)$ ，所以本算法的时间复杂度为 $O(n\log_2 n)$ 。

7.4 求解最优装载问题

【问题描述】 有 n 个集装箱要装上一艘载重量为 W 的轮船，其中集装箱 i ($1 \leq i \leq n$) 的重量为 w_i 。

不考虑集装箱的体积限制，现要选出尽可能多的集装箱装上轮船，使它们的重量之和不超过 W 。

【问题求解】第 5 章讨论了简单装载问题，采用回溯法选出尽可能少的集装箱个数。这里的最优解是选出尽可能多的集装箱个数，并采用贪心法求解。

当重量限制为 W 时， w_i 越小可装载的集装箱个数越多，所以采用优先选取重量轻的集装箱装船的贪心思路。

对 w_i 从小到大排序得到 $\{w_1, w_2, \dots, w_n\}$, 设最优解向量为 $x = \{x_1, x_2, \dots, x_n\}$, 显然, $x_1 = 1$, 则 $x' = \{x_2, \dots, x_n\}$ 是装载问题 $w' = \{w_2, \dots, w_n\}$, $W' = W - w_1$ 的最优解, 满足贪心最优子结构性质。

// 问题表示

```
int w[]={0,5,2,6,4,3};
```

```
int n=5,W=10;
```

// 求解结果表示

```
int maxw;
```

```
int x[MAXN];
```

void solve()

```
{  memset(x,0,sizeof(x));
```

```
    sort(w+1,w+n+1);
```

```
    maxw=0;
```

```
    int restw=W;
```

```
    for (int i=1;i<=n && w[i]<=restw;i++)
```

```
    {  x[i]=1;
```

```
        restw-=w[i];
```

```
        maxw+=w[i];
```

```
    }
```

```
}
```

// 各集装箱重量，不用下标 0 的元素

// 存放最优解的总重量

// 存放最优解向量

// 求解最优装载问题

// 初始化解向量

//w[1..n] 递增排序

// 剩余重量

// 选择集装箱 i

// 减少剩余重量

// 累计装载总重量

```
int w[]={0,5,2,6,4,3};  
int n=5,W=10;
```

// 各集装箱重量，不用下标 0 的元素



最优方案

选取重量为 2 的集装箱

选取重量为 3 的集装箱

选取重量为 4 的集装箱

总重量 = 9

【算法分析】 算法的主要时间花费在排序上，时间复杂度为 $O(n\log_2 n)$ 。

7.5 求解田忌赛马问题

【问题描述】二千多年前的战国时期，齐威王与大将田忌赛马。双方约定每人各出 300 匹马，并且在上、中、下三个等级中各选一匹进行比赛，由于齐威王每个等级的马都比田忌的马略强，比赛的结果可想而知。现在双方各 n 匹马，依次派出一匹马进行比赛，每一轮获胜的一方将从输的一方得到 200 银币，平局则不用出钱，田忌已知所有马的速度值并可以安排出场顺序，问他如何安排比赛获得的银币最多。

输入：输入包含多个测试用例，每个测试用例的第一行正整数 n ($n \leq 1000$) 马的数量，后两行分别是 n 个整数，表示田忌和齐威王的马的速度值。输入 $n=0$ 结束。

输出：每个测试用例输出一行，表示田忌获得的最多银币数。

输入样例:

3

92 83 71

95 87 74

2

20 20

20 20

2

20 19

22 18

0

样例输出:

200

0

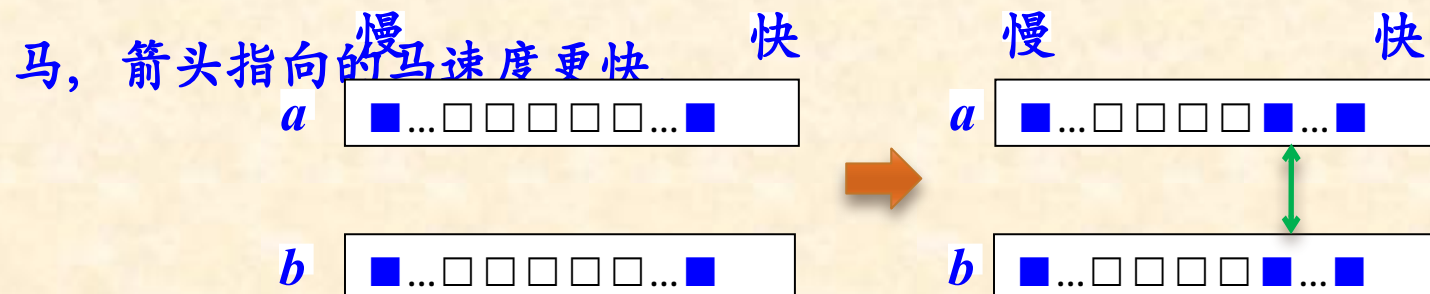
0

【问题求解】 田忌的马速度用数组 a 表示，齐威王的马速度用数组 b 表示，将 a 、 b 数组递增排序。采用常识性的贪心思路，分以下几种情况：

(1) 田忌最快的马比齐威王最快的马快即

$a[\text{righta}] > b[\text{rightb}]$ ，则两者比赛（两个最快的马比赛），田忌赢。

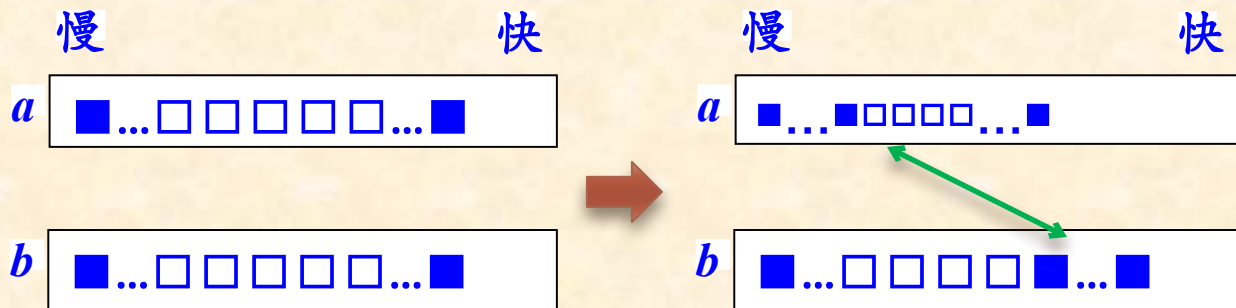
因为此时田忌最快的马一定赢，而选择与齐威王最快的马比赛对于田忌来说是最优的，下图中“■”代表已经比赛的马，“□”代表尚未比赛的马，箭头指向的马速度更快



$a[\text{righta}] > b[\text{rightb}] :$
 $\text{ans} += 200$

(2) 田忌最快的马比齐威王最快的马慢即

$a[\text{righta}] < b[\text{rightb}]$ ，则选择田忌最慢的马与齐威王最快的马比赛，田忌输。因为齐威王最快的马一定赢，而选择与田忌最慢的马比赛对于田忌来说是最优的。

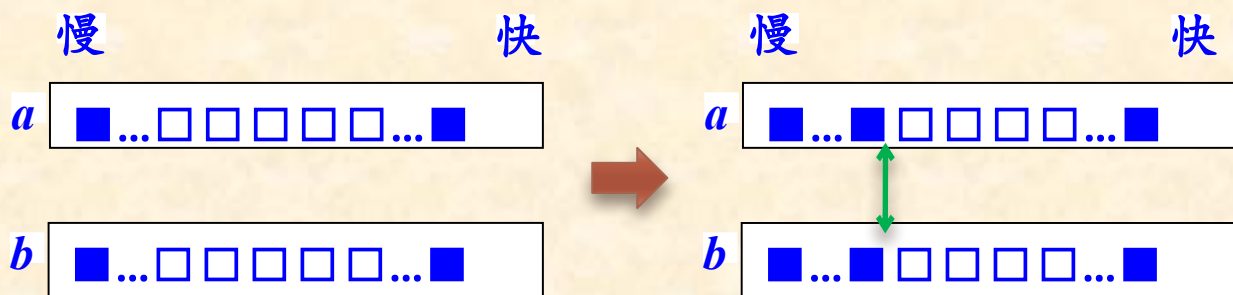


$a[\text{righta}] < b[\text{rightb}] : \text{ans} -$
 $= 200$

(3) 田忌最快的马与齐威王最快的马的速度相同即

$a[\text{righta}] = b[\text{rightb}]$ ，又分为以下 3 种情况：

① 田忌最慢的马比齐威王最慢的马快即 $a[\text{lefta}] > b[\text{leftb}]$ ，则两者比赛（两个最慢的马比赛），田忌赢。因为此时齐威王最慢的马一定输，而选择与田忌最慢的马比赛对于田忌来说是最优的。



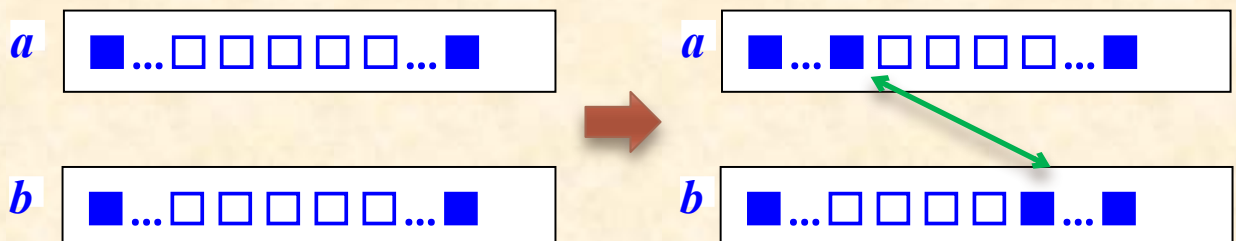
$a[\text{lefta}] > b[\text{leftb}] :$

$\text{ans} += 200$

② 田忌最慢的马比齐威王最慢的马慢，并且田忌最慢的马比齐威王最快的马慢，即 $a[\text{left}a] \leq b[\text{left}b]$ 且

$a[\text{left}a] < b[\text{right}b]$ ，则选择田忌最慢的马与齐威王最快的马比赛，田忌输。因为此时田忌最慢的马一定输，而选择与齐威王最快的马

比赛对于田忌来说是最优的。



$a[\text{left}a] \geq b[\text{left}b]$ 且 $a[\text{left}a] < b[\text{right}b]$:
ans -= 200

③ 其他情况, 即 $a[\text{righta}] = b[\text{rightb}]$ 且 $a[\text{lefta}] \leq b[\text{leftb}]$ 且 $a[\text{lefta}] \geq b[\text{rightb}]$, 则 $a[\text{lefta}] \geq b[\text{rightb}] = a[\text{righta}]$, 即 $a[\text{lefta}] = a[\text{righta}]$, $b[\text{leftb}] \geq a[\text{lefta}] = b[\text{rightb}]$, 即 $b[\text{leftb}] = b[\text{rightb}]$, 说明比赛区间的所以马的速度全部相同, 任何两匹马比赛都没有输赢。

上述过程看出每种情况对于田忌来说都是最优的, 因此最终获得的比赛方案也一定是最优的。

// 问题表示

int n;

int a[MAX];

int b[MAX];

// 求解结果表示

int ans;

int main()

{ while (true)

{ scanf("%d",&n);

if (n==0) break;

for (int i=0;i<n;i++)

scanf("%d",&a[i]);

for (int j=0;j<n;j++)

scanf("%d",&b[j]);

solve();

printf("%d\n",ans);

}

return 0;

}


```
void solve()                                // 求解算法
{
    sort(a,a+n);                            // 对 a 递增排序
    sort(b,b+n);                            // 对 b 递增排序
    ans=0;
    int lefta=0,leftb=0;
    int righta=n-1,rightb=n-1;

    while (lefta<=righta)                  // 比赛直到结束
    {
        if (a[righta]>b[rightb])
            // 田忌最快的马比齐威王最快的马快, 两者比赛
            {
                ans+=200;
                righta--;
                rightb--;
            }
        else if (a[righta]<b[rightb])
            // 田忌最快的马比齐威王最快的马慢
            {
                ans-=200;
                // 选择田忌最慢的马与比齐威王最快的马比赛
                lefta++;
                rightb--;
            }
    }
```

```
else    // 田忌最快的马与齐威王最快的马的速度相同
{  if  (a[lefta]>b[leftb])
    // 田忌最慢的马比齐威王最慢的马快，两者比赛
    {  ans+=200;
        lefta++;
        leftb++;
    }
    else
    {  if  (a[lefta]<b[rightb])
        // 否则，用田忌最慢的马与齐威王最快的马比赛
        ans-=200;
        lefta++;
        rightb--;
    }
}
}
```

【算法分析】 算法的主要时间花费在排序上，时间复杂度为 $O(n\log_2 n)$ 。

7.6 求解多机调度问题

【问题描述】 设有 n 个独立的作业 $\{1, 2, \dots, n\}$ ，由 m 台相同的机器 $\{1, 2, \dots, m\}$ 进行加工处理，作业 i 所需的处理时间为 t_i ($1 \leq i \leq n$)，每个作业均可在任何一台机器上加工处理，但未完工前不允许中断，任何作业也不能拆分成更小的子作业。

多机调度问题要求给出一种作业调度方案，使所给的 n 个作业在尽可能短的时间内由 m 台机器加工处理完成。

【问题求解】 贪心法求解多机调度问题的贪心策略是最长处理时间作业优先，即把处理时间最长的作业分配给最先空闲的机器，这样可以保证处理时间长的作业优先处理，从而在整体上获得尽可能短的处理时间。

按照最长处理时间作业优先的贪心策略，当 $m \geq n$ 时，只要将机器 i 的 $[0, t_i)$ 时间区间分配给作业 i 即可；

当 $m < n$ 时，首先将 n 个作业依其所需的处理时间从大到小排序，然后依此顺序将作业分配给空闲的处理机。

例如，有 7 个独立的作业 {1, 2, 3, 4, 5, 6, 7}，由 3 台机器 {1, 2, 3} 加工处理，各作业所需的处理时间如下：

作业编号	1	2	3	4	5	6	7
作业的处理时间	2	14	4	16	6	5	3

采用贪心法求解的过程如下：

(1) 7 个作业按处理时间递减排序，其结果如下表所示

作业编号	4	2	5	6	3	7	1
作业的处理时间	16	14	6	5	4	3	2

(2) 先将排序后的前3个作业分配给3台机器。此时机器的分配情况为 $\{\{4\}, \{2\}, \{5\}\}$ ，对应的总处理时间为 $\{16, 14, 6\}$ 。

作业编号	4	2	5	6	3	7	1
作业的处理时间	16	14	6	5	4	3	2

机器
1



作业 4:16

机器
2



作业 2:14

机器
3



作业 5:6

(3) 分配余下的作业:

作业编号	4	2	5	6	3	7	1
作业的处理时间	16	14	6	5	4	3	2

机器
1



作业 4 , 总时间
16

机器
2



作业 2、7 , 总时间
17

机器
3



作业 5、6、3、1 , 总时间
17

作业
调度
方案

// 问题表示

int n=7;

int m=3;

struct NodeType

// 优先队列结点类型

{ int no;

// 作业序号

int t;

// 执行时间

int mno;

// 机器序号

bool operator<(const NodeType &s) const

{ return t>s.t; }

// 按 t 越小越优先出队

};

struct NodeType A[]={ {1,2}, {2,14}, {3,4}, {4,16}, {5,6}, {6,5},
{7,3}};

```

void solve() // 求解多机调度问题
{
    NodeType e;
    if (n<=m)
    {
        printf(" 为每一个作业分配一台机器 \n");
        return;
    }
    sort(A,A+n); // 按 t 递减排序
    priority_queue<NodeType> qu; // 小根堆
    for (int i=0;i<m;i++) // 先分配 m 个作业，每台机器一个作业
    {
        A[i].mno=i+1; // 作业对应的机器编号
        printf("    给机器 %d 分配作业 %d，执行时间为 %2d，占用时间段 : [%d, %d]\n",
                A[i].mno,A[i].no,A[i].t,0,A[i].t);
        qu.push(A[i]);
    }
    for (int j=m;j<n;j++) // 分配余下作业
    {
        e=qu.top(); qu.pop(); // 出队 e
        printf("    给机器 %d 分配作业 %d，执行时间为 %2d，占用时间段 : [%d, %d]\n",
                e.mno,A[j].no,A[j].t,e.t,e.t+A[j].t);
        e.t+=A[j].t;
        qu.push(e); // e 进队
    }
}

```

【算法分析】 排序的时间复杂度为 $O(n\log_2 n)$ ，两次 for 循环的时间合起来为 $O(n)$ ，所以本算法的时间复杂度为 $O(n\log_2 n)$ 。

7.7 哈夫曼编码

【问题描述】 设要编码的字符集为 $\{d_1, d_2, \dots, d_n\}$ ，它们出现的频率为 $\{w_1, w_2, \dots, w_n\}$ ，应用哈夫曼树构造最优的不等长的由 0、1 构成的编码方案。

【问题求解】 先构建以这个 n 个结点为叶子结点的哈夫曼树，然后由哈夫曼树产生各叶子结点对应字符的哈夫曼编码。

哈夫曼树 (Huffman Tree) 的定义：设二叉树具有 n 个带权值的叶子结点，从根结点到每个叶子结点都有一个路径长度。从根结点到各个叶子结点的路径长度与相应结点权值的乘积的和称为该二叉树的带权路径长度，记作：

$$WPL = \sum_{i=1}^n w_i \times l_i$$

由 n 个叶子结点可以构造出多种二叉树，其中具有最小带权路径长度的二叉树称为**哈夫曼树**（也称最优树）。

构造一棵哈夫曼树的方法如下：

(1) 由给定的 n 个权值 $\{w_1, w_2, \dots, w_n\}$ 构造 n 棵只有一个叶子结点的二叉树，从而得到一个二叉树的集合 $F=\{T_1, T_2, \dots, T_n\}$ 。

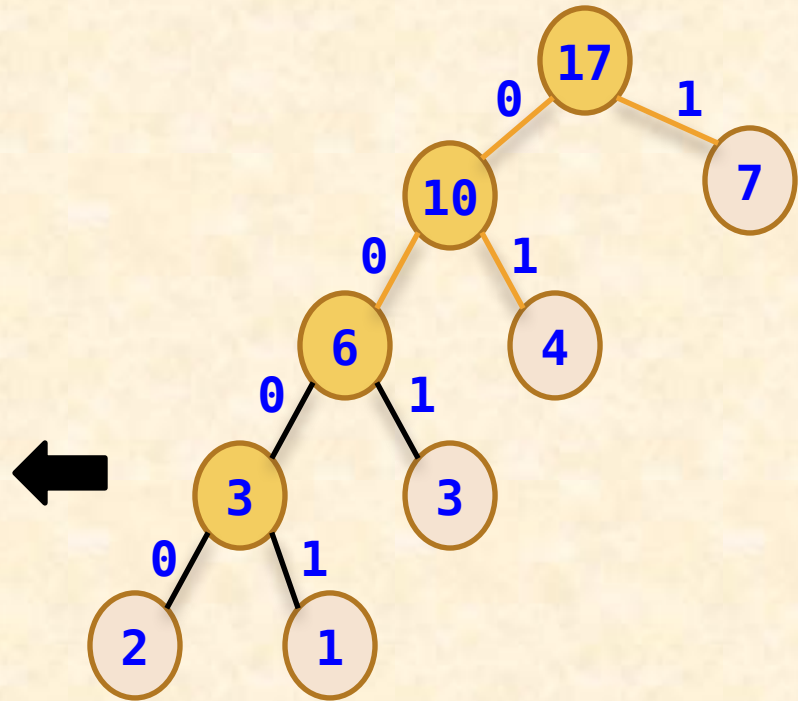
(2) 在 F 中选取根结点的权值最小和次小的两棵二叉树作为左、右子树构造一棵新的二叉树，这棵新的二叉树根结点的权值为其左、右子树根结点权值之和。即合并两棵二叉树为一棵二叉树。

(3) 重复步骤 (2)，当 F 中只剩下一棵二叉树时，这棵二叉树便是所要建立的哈夫曼树。

例如，给定的 $a \sim e$ 的 5 个字符，它们的权值集合为 $W=\{4, 2, 1, 7, 3\}$ ，构造哈夫曼树的过程如下。

(1) $\{4, 2, 1, 7, 3\} \Rightarrow \{4, 3, 7, 3\}$
(2) $\{4, 3, 7, 3\} \Rightarrow \{4, 6, 7\}$
(3) $\{4, 6, 7\} \Rightarrow \{10, 7\}$
(4) $\{10, 7\} \Rightarrow \{17\}$

b(2) : 0000 c(1) : 0001
e(3) : 001 a(4) : 01
d(7) : 1
WPL = (2+1)*4 + 3*3 + 4*2 + 7*1 =
36




```
int n;  
struct HTreeNode  
{   char data;  
    int weight;  
    int parent;  
    int lchild;  
    int rchild;  
};  
HTreeNode ht[MAX];  
map<char,string> htcode;
```

```
// 哈夫曼树结点类型  
// 字符  
// 权值  
// 双亲的位置  
// 左孩子的位置  
// 右孩子的位置
```

```
struct NodeType  
{   int no;  
    char data;  
    int weight;  
    bool operator<(const NodeType &s) const  
    {  
        return s.weight<weight;  
    }  
};
```

```
// 存放哈夫曼树  
// 存放哈夫曼编码  
  
// 优先队列结点类型  
// 对应哈夫曼树 ht 中的位置  
// 字符  
// 权值  
// 用于创建小根堆
```

```

void CreateHTree()                                // 构造哈夫曼树
{
    NodeType e,e1,e2;
    priority_queue<NodeType> qu;
    for (int k=0;k<2*n-1;k++)                    // 设置所有结点的指针域
        ht[k].lchild=ht[k].rchild=ht[k].parent=-1;
    for (int i=0;i<n;i++)                        // 将 n 个结点进队 qu
    {
        e.no=i; e.data=ht[i].data;
        e.weight=ht[i].weight; qu.push(e);
    }

    for (int j=n;j<2*n-1;j++)                    // 构造哈夫曼树的 n-1 个非
叶子结点
    {
        e1=qu.top(); qu.pop();                  // 出队权值最小的结点 e1
        e2=qu.top(); qu.pop();                  // 出队权值次小的结点 e2
        ht[j].weight=e1.weight+e2.weight;        // 构造哈夫曼树的非叶子结
点 j
        ht[j].lchild=e1.no;
        ht[j].rchild=e2.no;
        ht[e1.no].parent=j;                     // 修改 e1.no 的双亲为结点 j
        ht[e2.no].parent=j;                     // 修改 e2.no 的双亲为结点 j
        e.no=j;                                 // 构造队列结点 e
        e.weight=e1.weight+e2.weight;
        qu.push(e);
    }
}

```

```
void CreateHCode()
```

```
// 构造哈夫曼编码
```

```
{  string code;  
   code.reserve(MAX);
```

```
   for (int i=0;i<n;i++)
```

```
// 构造叶子结点 i 的哈夫曼编码
```

```
   {  code="";
```

```
       int curno=i;
```

```
       int f=ht[curno].parent;
```

```
       while (f!=-1)
```

```
// 循环到根结点
```

```
       {  if (ht[f].lchild==curno)  //curno 为双亲 f 的左孩子
```

```
           code='0'+code;
```

```
           else
```

```
//curno 为双亲 f 的右孩子
```

```
           code='1'+code;
```

```
           curno=f; f=ht[curno].parent;
```

```
       }
```

```
       htcode[ht[i].data]=code;  // 得到 ht[i].data 字符的哈夫曼
```

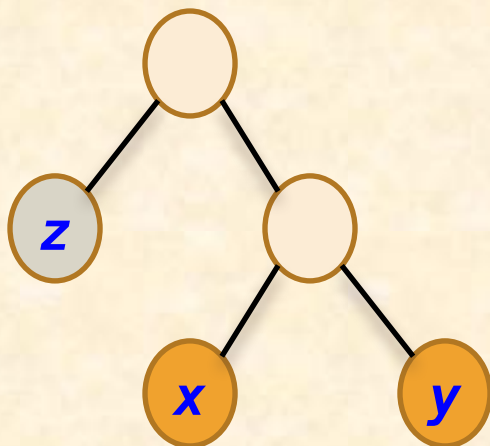
```
编码
```

```
   }
```

```
}
```

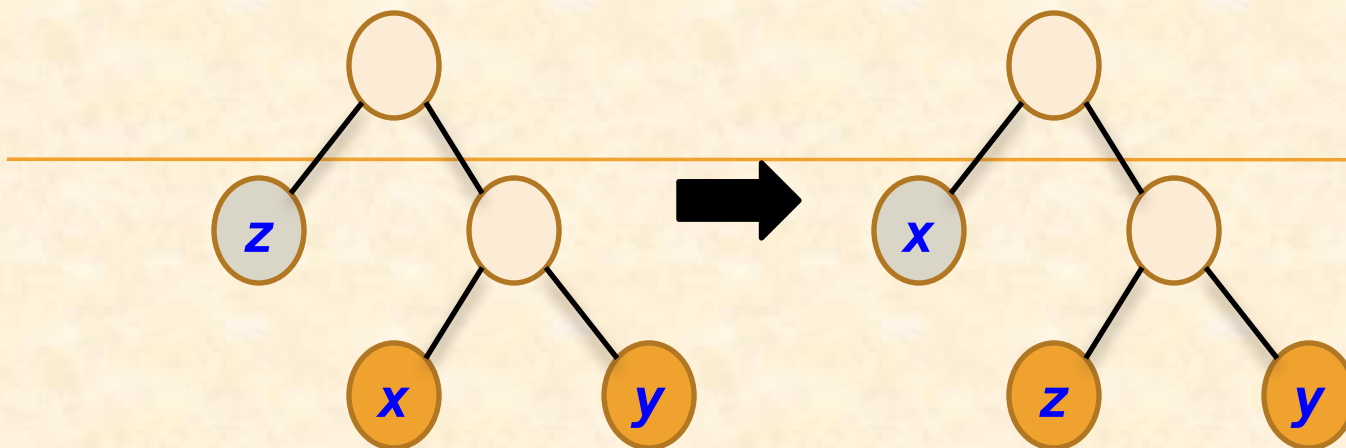
【算法证明】先讨论两个命题及其证明过程。

命题 1： 两个最小权值字符对应的结点 x 和 y 必须是哈夫曼树中最深的两个结点且它们为兄弟。



证明： 假设 x 结点在哈夫曼树（最优树）中不是最深的，那么存在一个结点 z ，有 $w_z > w_x$ ，但它比 x 深，即 $l_z > l_x$ 。

此时 x 和 z 的带权和为 $w_x \times l_x + w_z \times l_z$ 。



如果交换 x 和 z 结点的位置，其他不变

□ 交换后的带权和为 $w_x \times l_z + w_z \times l_x$ 。

有: $w_x \times l_z + w_z \times l_x < w_x \times l_x + w_z \times l_z$

这是因为 $w_x \times l_z + w_z \times l_x - (w_x \times l_x + w_z \times l_z)$

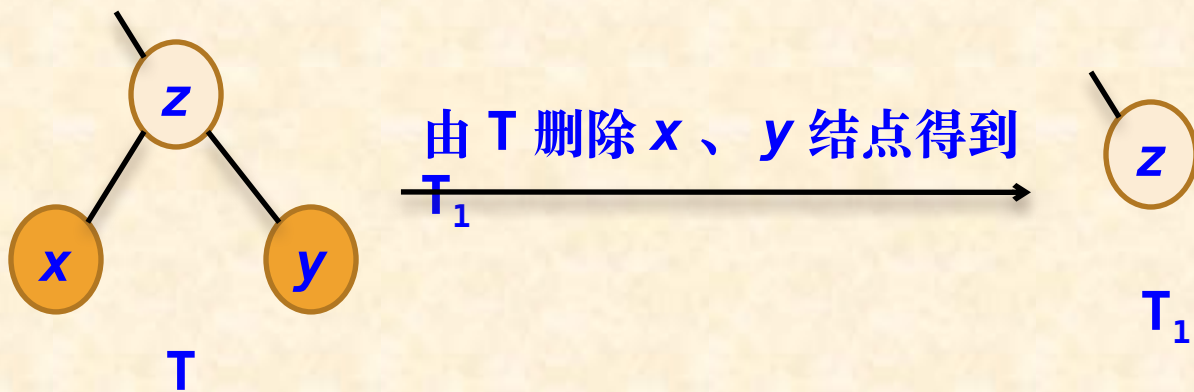
$$= w_x(l_z - l_x) - w_z(l_z - l_x)$$

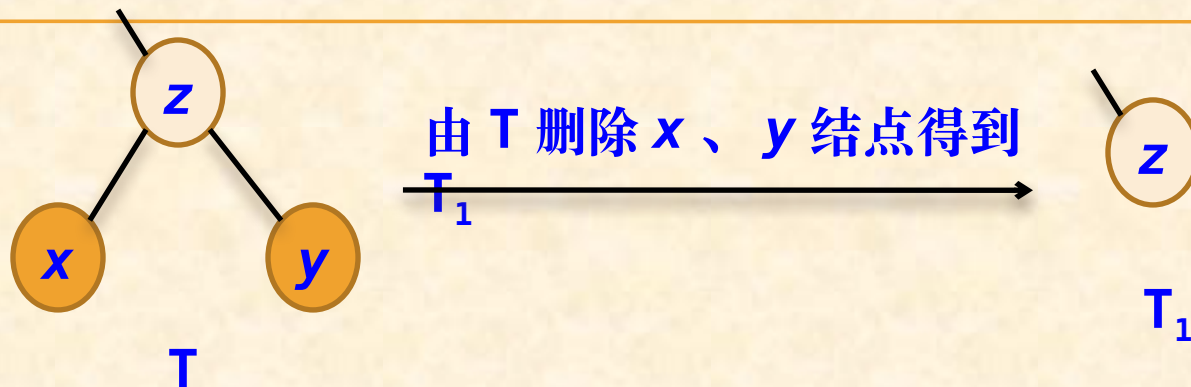
$$= (w_x - w_z)(l_z - l_x) < 0 \quad (\text{由前面所设有 } w_z > w_x \text{ 和}$$

$l_z > l_x$)。

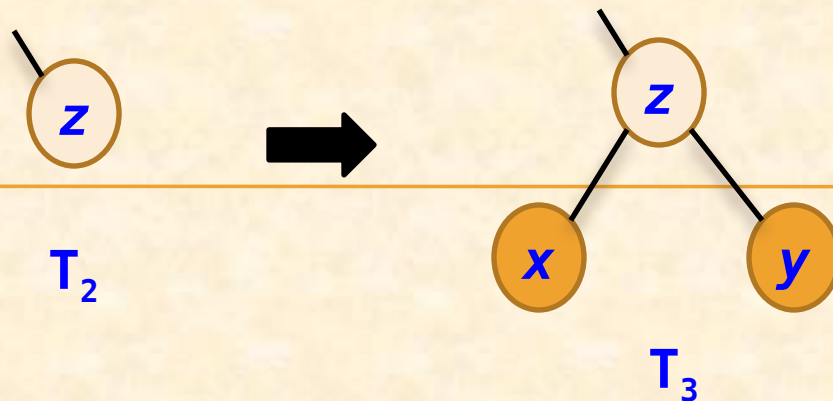
这就与交换前的树是最优树的假设矛盾。所以上述命题成立。

命题 2： 设 T 是字符集 C 对应的一棵哈夫曼树，结点 x 和 y 是兄弟，它们的双亲为 z ，显然有 $w_z = w_x + w_y$ ，现删除结点 x 和 y ，让 z 变为叶子结点，那么这棵新树 T_1 一定是字符集 $C_1 = C - \{x, y\} \cup \{z\}$ 的最优树。





证明： 设 T 和 T_1 的带权路径长度分别为 $WPL(T)$ 和 $WPL(T_1)$ ，则有：
$$WPL(T) = WPL(T_1) + w_x + w_y$$
这是因为 $WPL(T_1)$ 含有 T 中除 x 、 y 外的所有叶子结点的带权路径长度和，另加上 z 的带权路径长度。



假设 T_1 不是最优的，则存在另一棵树 T_2 ，有：

$$WPL(T_2) < WPL(T_1)$$

由于 $z \in C_1$ ，则 z 在 T_2 中一定是一个叶子结点。若将 x 和 y 加入 T_2 中作为结点 z 的左、右孩子，则得到表示字符集 C 的前缀树 T_3

且有：

$$WPL(T_3) = WPL(T_2) + w_x + w_y$$

由前面几个式子看到

$$WPL(T_3) = WPL(T_2) + w_x + w_y < WPL(T_1) + w_x + w_y = WPL(T)$$

这与 T 为 C 的哈夫曼树的假设矛盾。本命题即证。

命题 1 说明该算法满足贪心选择性质，即通过合并来构造一棵哈夫曼树的过程可以从合并两个权值最小的字符开始。

命题 2 说明该算法满足最优子结构性质，即该问题的最优解包含其子问题的最优解。所以采用哈夫曼树算法产生的树一定是一棵最优树。

【算法分析】上述算法采用了小根堆，因为从堆中删除两个结点（权值最小的两个二叉树根结点）和加入一个新结点的时间复杂度是 $O(\log_2 n)$ ，这样修改后构造哈夫曼树算法的时间复杂度为 $O(n\log_2 n)$ 。

算法导论 p239 带惩罚的任务调度算法

单处理器上带截止时间和惩罚的单位时间任务调度问题有以下输入:

(1) n 个单位时间任务的集合 $S=\{a_1, a_2, \dots, a_n\}$;

(2) n 个整数截止时间 $d_1, d_2, \dots, d_n$, 每个 d_i 满足 $1 \leq d_i \leq n$, 期望任务 a_i 在时间 d_i 之前完成。

(3) n 个非负权重或者惩罚 $w_1, w_2, \dots, w_n$, 若任务 a_i 在时间 d_i 之前没有完成, 就会受到 w_i 这么多的惩罚, 如果任务在截止时间之前完成, 则不会受到惩罚。

贪心选择方法：惩罚越大越优先执行！

i	1	2	3	4	5	6	7
d_i	4	2	4	3	1	4	6
w_i	70	60	50	40	30	20	10

用 **sum** 累计做过作业的时间（初始化为 0）

$d_i > \text{sum} \Rightarrow$ 可以做 $d_i \leq \text{sum} \Rightarrow$ 不能做

i	1	2	3	4	5	6	7
前 sum	0	1	2	3	3	3	4
d_i	4	2	4	3	1	4	6
w_i	70✓	60✓	50✓	40✗	30✗	20✓	10✓
flag	1	2	3	3	3	4	5
后 sum							

惩罚

$= 40 + 30 = 70$

```
void solve()  
{  memset(flag,0,sizeof(flag));  
    sort(A,A+n);  
    int sum=0;  
    for (int i=0;i<n;i++)  
    {  
        if (A[i].d>sum)  
        {  
            flag[i]=true;  
            sum++;  
        }  
    }  
}
```

// 求解问题

//flag 数组初始化

// 按逾期惩罚分递减排序

// 累计做过作业的时间

// 选择做 A[i] 作业

// 时间加 1

Hulu 2013 北京地区校招笔试题

hulu (“葫芦”和“互录”) 是一家美国的视频网站。该网站由美国国家广播环球公司 (NBC Universal) 和福克斯在 2007 年 3 月共同注册成立。

该网站在洛杉矶、纽约、北京均设有办事处。

并于 2007 年十月获得了私人股权投资公司普罗维登斯股本合伙人一亿美元的风险投资基金。2011 年 7 月 19 日微软放弃竞购 Hulu , 雅虎以 20 亿美元胜出。

填空题：

1、中序遍历二叉树，结果为 ABCDEFGH，后序遍历结果为 ABEDCHGF，先序遍历结果为（ ）。

2、对字符串 HELLO_HULU 中的字符进行二进制编码，使得字符串的编码长度尽可能短，最短长度为（ ）。

3、对长度 12 的有序数组进行二分查找，目标等概率出现在数组的每个位置上，则平均比较次数为（ ）。

4、一副扑克（去王），每个人随机的摸两张，则至少需要（ ）人摸牌，才能保证有两个人抽到同样的花色。

令 A、B、C、D 依次代表扑克牌中的四种花色，随机抽取的两张牌的花色组合有 10 种，根据抽屉原理，则至少 11 个人抽时，才能保证有两个人抽到同样的花色。

5、x 个小球中有唯一一个球较轻，用天平秤最少称量 y 次能找出这个较轻的球，写出 y 和 x 的函数表达式 $y=f(x)$ 。（ ）

… …（共 11 题）

大题:

- 1、 n 个数，找出其中最小的 k 个数，写出代码，要求最坏情况下的时间复杂度不能高于 $O(n\log_2 k)$ 。
- 2、写程序输出 8 皇后问题的所有排列，要求使用非递归的深度优先遍历。
- 3、有 n 个作业， $a_1, a_2, \dots, a_n$ ，作业 a_j 的处理时间为 t_j ，产生的效益为 p_j ，最后完成期限为 d_j ，作业一旦被调度则不能中断，如果作业 a_j 在 d_j 前完成，则获得效益 p_j ，否则无效益。给出最大化效益的作业调度算法。


```

#include <stdio.h>
#include <algorithm>
using namespace std;
#define MAXN 51
int n=3;
int t[] = {0,1,4,1};           // 下标 0 不用
int d[] = {0,5,4,5};
int p[] = {0,2,8,6};
struct Action
{
    int t;           // 处理时间
    int d;           // 完成期限
    int p;           // 效益
    bool operator < (const Action t) const
    {
        return p>t.p; // 按效益递减排序
    }
};
Action A[MAXN];           // 求解结果表示
int bestp=0;              // 最大的效益

```

```

void solve()
{
    sort(A+1,A+n+1);
    int sum=0;
    for (int i=1;i<=n;i++)
    {
        if (A[i].d>sum)
        {
            bestp+=A[i].p;
            sum+=A[i].t;
        }
    }
}

void main()
{
    for (int i=1;i<=n;i++) // 产生 A 数组元素
    {
        A[i].t=t[i];
        A[i].d=d[i];
        A[i].p=p[i];
    }
    solve();
    printf("%d\n",bestp); // 输出 14
}

```

// 求解算法

// 按效益递减排序

// 初始化处理作业的时间为 0

// 累计处理作业的效益

// 累计处理时间

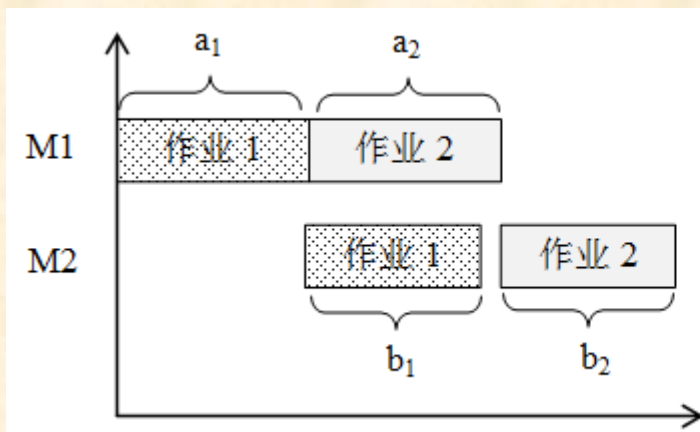
7.8 求解流水作业调度问题

【问题描述】有 n 个作业（编号为 $1 \sim n$ ）要在由两台机器 M1 和 M2 组成的流水线上完成加工。每个作业加工的顺序都是先在 M1 上加工，然后在 M2 上加工。M1 和 M2 加工作业 i 所需的时间分别为 a_i 和 b_i ($1 \leq i \leq n$)。

流水作业调度问题要求确定这 n 个作业的最优加工顺序，使得从第一个作业在机器 M1 上开始加工，到最后一个作业在机器 M2 上加工完成所需的时间最少。可以假定任何作业一旦开始加工，就不允许被中断，直到该作业被完成，即非优先调度。

【问题求解】 采用归纳思路。当只有一个作业 (a_1 , b_1), 显然最少时间 $T_{\min}=a_1+b_1$ 。

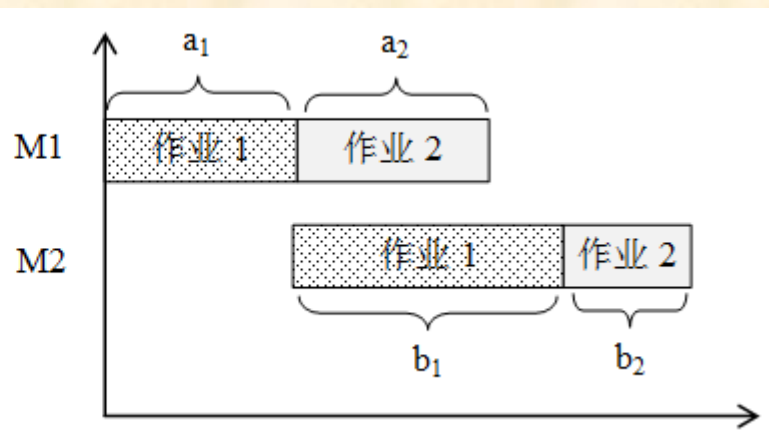
当有两个作业 (a_1 , b_1) 和 (a_2 , b_2) 时, 若 (a_1 , b_1) 在前 (a_2 , b_2) 在后执行:



(a) 作业 2 在 M2 上执行没有等待的情况

$$T_{\min}=a_1+b_1+c_1+d_1-b_1$$

($b_1 < a_2$)



(b) 作业 2 在 M2 上执行有等待的情况

$$T_{\min}=a_1+b_1+c_1+d_1-a_2$$

($a_2 < b_1$)

$$T_{\min}=a_1+b_1+a_2+b_2-\min(a_2, b_1)。$$

若 (a_2, b_2) 在前 (a_1, b_1) 在后执行, 可以求出最少时间

$$T_{\min} = a_2 + b_2 + a_1 + b_1 - \min(b_2, a_1)$$



将两种执行顺序合并起来有:

$$T_{\min} = a_1 + b_1 + a_2 + b_2 - \max(\min(a_2, b_1), \min(a_1, b_2))$$

由此可以得到一个贪心的性质:

对于 (a_1, b_1) 和 (a_2, b_2) 中的元素

■ 若 $\min(a_1, b_2) \leq \min(a_2, b_1)$, 则 (a_1, b_1) 放在 (a_2, b_2) 前面;

■ 反之, 若 $\min(a_2, b_1) < \min(a_1, b_2)$ (a_2, b_2) 放在 (a_1, b_1) 前面。



让 (a, b) 中 a 比较小的尽可能先执行, (a, b) 中 b 比较小的尽可能后执行!

Johnson 贪心算法。其步骤如下：

(1) 把所有作业按 $M1$ 、 $M2$ 的时间分为两组， $a[i] \leq b[i]$ 对应第 1 组 $N1$ ， $a[i] > b[i]$ 对应第 0 组 $N2$ 。

(2) 将 $N1$ 的作业按 $a[i]$ 递增排序， $N2$ 的作业按 $b[i]$ 递减排序。

(3) 按顺序先执行 $N1$ 的作业，再执行 $N2$ 的作业，得到的就是耗时最少的最优调度方案。

实际上， N_2 的作业也按 $b[i]$ 递增排序，从后面向前面顺序执行

示例: $n=4$

编号	1	2	3	4
M1	5	12	4	8
M2	6	2	14	7

(1) 把所有作业按 M1、M2 的时间分为两组, $a[i] \leq b[i]$ 对应第 1 组 N1, $a[i] > b[i]$ 对应第 0 组 N2。

组号	1	0	1	0
时间 time	5	2	4	7

(2) 将 N1 (组号 =1, $a[i] \leq b[i]$) 的作业按 $a[i]$ (用时间 **time** 存放) 递增排序, N2 (组号 =1, $a[i] > b[i]$) 的作业按 $b[i]$ (用时间 **time** 存放) 递增排序。

编号	1	2	3	4
M1	5	12	4	8
M2	6	2	14	7
组号	1	0	1	0
时间 time	5	2	4	7



编号	2	3	1	4
M1	12	4	5	8
M2	2	14	6	7
组号	0	1	1	0
时间 time	2	4	5	7

(3) 按顺序先执行 N1 的作业 (顺序), 再执行 N2 的作业 (反序), 得到的就是耗时最少的最优调度方案。

编号	2	3	1	4
M1	12	4	5	8
M2	2	14	6	7
组号	0	1	1	0
时间 time	2	4	5	7



编号	3	1	4	2
M1	4	5	8	12
M2	14	6	7	2
组号	1	1	0	0
时间 time	4	5	7	2

最优调度方案: 3 1 4 2

求在最优调度下总时间，用 **f1** 累计 M1 上的执行时间（初始时 **f1=0**）；
f2 累计 M2 上的执行时间（初始时 **f2=0**），最终 **f2** 即为最优调度下的消耗
总时间。

对于最优调度方案 **best**，用 *i* 扫描 **best** 的元素，**f1** 和 **f2** 的计算如
下：

$f1 = f1 + a[best[i]]$

编号	3	1	4	2
M1	4	5	8	12
M2	14	6	7	2
组号	1	1	0	0
时间 time	4	5	7	2

f1=f2=0

作业 3： $f1=0+4=4$ ， $f2=\max\{4,0\}+14=18$

作业 1： $f1=4+5=9$ ， $f2=\max\{9,18\}+6=24$

作业 4： $f1=9+8=17$ ， $f2=\max\{17,24\}+7=31$

作业 2： $f1=17+12=29$ ， $f2=\max\{29,31\}+2=33$

其实现采用如下结构体数组 c:

```
struct NodeType
{
    int no;                // 作业序号
    bool group;            // 1 代表第一组 N1, 0 代表第二组 N2
    int time;              // a, b 的最小时间
    bool operator<(const NodeType &s) const
    {
        return time<s.time; // 用于按 time 递增排序
    }
};
```

如

编号	1	2	3	4
M1	5	12	4	8
M2	6	2	14	7
组号	1	0	1	0
时间 time	5	2	4	7

// 问题表示

```
int n=4;
```

```
int a[N]={5,12,4,8};
```

// 对应 M1 的时间

```
int b[N]={6,2,14,7};
```

// 对应 M2 的时间

```
struct NodeType
```

```
{ int no;
```

// 作业序号

```
    bool group;
```

//1 代表第一组 N1,0 代表

第二组 N2

```
    int time;
```

//a,b 的最小时间

```
    bool operator<(const NodeType &s) const
```

```
{
```

```
    return time<s.time;    // 按 time 递增排序
```

```
}
```

```
};
```

// 求解结果表示

```
int best[N];
```

// 最优调度序列

```
int solve()                // 求解流水作业调度问题
{  int i,j,k;
   NodeType c[N];
   for(i=0;i<n;i++) //n 个作业中，求出每个作业的最小加工时间
   {  c[i].no=i;
      c[i].group=(a[i]<=b[i]);
                                     //a[i]<=b[i] 对应第 1 组 N1,a[i]>b[i] 对应第 0
组 N2
      c[i].time=a[i]<=b[i]?a[i]:b[i];
                                     // 第 1 组存放 a[i], 第 0 组存放 b[i]
   }
   sort(c,c+n);             //c 元素按 time 递增排序
```

```

j=0; k=n-1;
for(i=0;i<n;i++)          // 扫描 c 所有元素，产生最优调度方案
{   if(c[i].group==1)      // 第 1 组，按 time 递增排列放在 best 的前面
    best[j++]=c[i].no;
    else                  // 第 0 组，按 time 递减排列放到 best 的后面
    best[k--]=c[i].no;
}

int f1=0;                  // 累计 M1 上的执行时间
int f2=0;                  // 最优调度下的消耗总时间
for(i=0;i<n;i++)
{   f1+=a[best[i]];
    f2=max(f2,f1)+b[best[i]];
}
return f2;
}

```



```
void main()
{   printf(" 求解结果 \n");
    printf("      总时间 : %d\n",solve());
    printf("      调度方案 : ");
    for(int i=0;i<n;i++)
        printf("%d ",best[i]+1);
    printf("\n");
}
```

编号	1	2	3	4
M1	5	12	4	8
M2	6	2	14	7



求解结果

总时间：33

调度方案：3 1 4 2

【算法分析】 算法的主要时间花费在排序上，所以时间复杂度为 $O(n\log_2 n)$ 。比采用回溯法和分枝限界法求解更高效。

